CS 419

Secure Document Sharing System

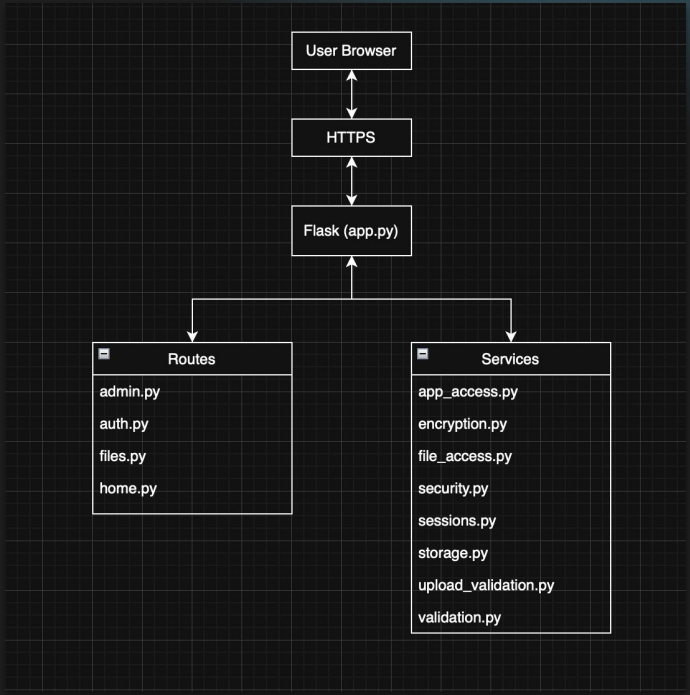
By: Krishanth Babu and Yat Wang

Project Overview

- Secure Document upload/download with encryption
- User registration and authentication
- RBAC with Admin, Users, and Guests
- Ability to share documents with specific users
- Audit Logging
- Backend: Python/Flask
- Encryption: bcrypt
- Storage: JSON Files

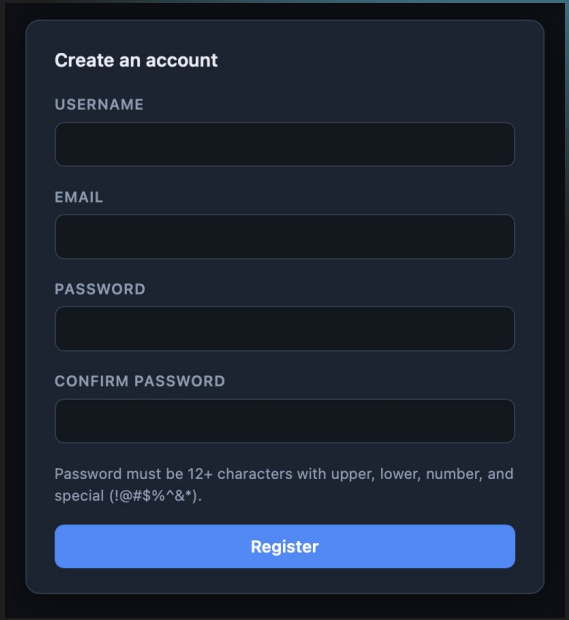
Security Architecture

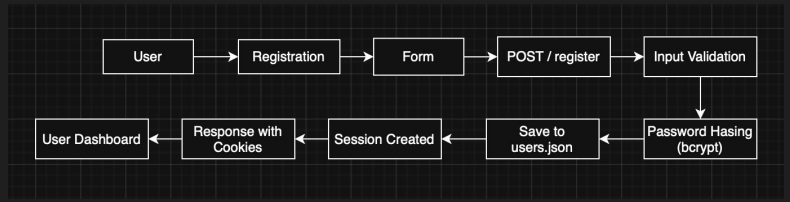
- Layered defense in depth approach
- From the start we enforce HTTPS
- Our routes handle the bulk of authentication, files, and admin functions.
- These later call into our services group which handles everything from encryption and validation to access control and session management



Authentication

- Validates usernames to include only 3-20 characters with only alphanumeric characters.
- Passwords must be 12+ characters, with upper lower, number and special
- Rate Limiting 10 attempts per IP per minute
- Account Lockout after 5 failed attempts
- Successful login creates a secure token





Encryption & Access Control

- All uploaded files are encrypted
- Encryption key is stored in a separate file
- RBAC with 3 different roles (Guest, User, Admin)
  - Users by default are Guests
- Includes File Level Permissions
  - Enforced via Server Side

Input Validation & Headers

- Inputs:
  - Length restricted
  - Sanitized using html.escape()
- Uploaded Files:
  - Extensions are checked (whitelist)
  - MIME sniffed for potential malware using filetype.guess()

- Headers enforce HTTPS, block app permissions, hide routes across sites and server info, enable XSS protection, block MIME sniffing, and prevent clickjacking

```
# HSTS (HTTPS only)
    response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
    response.headers['Permissions-Policy'] = ('geolocation=(), microphone=(), camera=()')
# Referrer Policy
    response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
# Hide server information
    response.headers.pop('Server', None)
    response.headers.pop('server', None)
# XSS Protection (legacy browsers)
    response.headers['X-XSS-Protection'] = '1; mode=block'
# Prevent MIME type sniffing
    response.headers['X-Content-Type-Options'] = 'nosniff'
# Prevent clickjacking
    response.headers['X-Frame-Options'] = 'DENY'
```

Challenges & Lessons

- Challenges:
  - Appropriate logging for each action
  - Identifying and preventing potential attacks
  - Testing defense mechanisms

- Lessons:
  - Defining clear roles and permissions before implementation makes logging more intuitive
  - Using trusted libraries helps save a lot of time during secure implementations development
  - Defense in depth even at application level provides a clear structure for testing